

```
import os

from dataclasses import dataclass
from typing import Optional
from dotenv import load_dotenv

load_dotenv()

@dataclass
class Config:
    """Configuration for the RAG application"""

    # Embedding configuration
    EMBEDDING_MODEL: str = os.getenv("EMBEDDING_MODEL", "all-MiniLM-L6-v2")
    EMBEDDING_DIM: int = int(os.getenv("EMBEDDING_DIM", "384"))

    # Chunking configuration
    CHUNK_SIZE: int = int(os.getenv("CHUNK_SIZE", "512"))
    CHUNK_OVERLAP: int = int(os.getenv("CHUNK_OVERLAP", "50"))

    # Retrieval configuration
    TOP_K: int = int(os.getenv("TOP_K", "5"))

    # Vector store configuration
    VECTOR_DB_PATH: str = os.getenv("VECTOR_DB_PATH", "data/vectordb")

    # LLM configuration
    LLM_MODEL: str = os.getenv("LLM_MODEL", "gpt-3.5-turbo")
    OPENAI_API_KEY: Optional[str] = os.getenv("OPENAI_API_KEY")
```

```
# Logging

LOG_LEVEL: str = os.getenv("LOG_LEVEL", "INFO")


# Evaluation

EVALUATION_QUESTIONS_PATH: str =
os.getenv("EVALUATION_QUESTIONS_PATH", "data/evaluation_questions.json")


@classmethod
def from_env(cls) -> "Config":
    """Create Config from environment variables"""
    return cls()


def validate(self) -> bool:
    """Validate configuration"""
    if self.OPENAI_API_KEY is None:
        print("Warning: OPENAI_API_KEY not set. LLM features will not work.")
        return False
    return True


config = Config.from_env()


import numpy as np
from sentence_transformers import SentenceTransformer
from typing import List, Optional
import logging


logger = logging.getLogger(__name__)


class EmbeddingGenerator:
    """Generate embeddings for text chunks"""
```

```
def __init__(self, model_name: str = "all-MiniLM-L6-v2"):
```

```
    """
```

```
    Initialize the embedding generator
```

```
    Args:
```

```
        model_name: Name of the sentence transformer model
```

```
    """
```

```
    self.model_name = model_name
```

```
    self.model = None
```

```
    self.dimension = 384 # all-MiniLM-L6-v2 dimension
```

```
def load_model(self):
```

```
    """Lazy load the model"""
```

```
    if self.model is None:
```

```
        logger.info(f"Loading embedding model: {self.model_name}")
```

```
        self.model = SentenceTransformer(self.model_name)
```

```
        self.dimension = self.model.get_sentence_embedding_dimension()
```

```
        logger.info(f"Model loaded. Dimension: {self.dimension}")
```

```
    return self.model
```

```
def embed(self, texts: List[str]) -> np.ndarray:
```

```
    """
```

```
    Generate embeddings for a list of texts
```

```
    Args:
```

```
        texts: List of text strings
```

```
    Returns:
```

numpy array of embeddings

"""

if not texts:

return np.array([])

model = self.load_model()

embeddings = model.encode(texts, convert_to_numpy=True)

return embeddings

def embed_single(self, text: str) -> np.ndarray:

"""

Generate embedding for a single text

Args:

text: Text string

Returns:

numpy array of embedding

"""

return self.embed([text])[0]

@property

def embedding_dimension(self) -> int:

"""Get the embedding dimension"""

if self.model is None:

self.load_model()

return self.dimension

import sqlite3

```

import json
import numpy as np
import faiss
from typing import List, Dict, Any, Optional, Tuple
from pathlib import Path
import hashlib
import logging

logger = logging.getLogger(__name__)

class VectorStore:
    """
    FAISS + SQLite vector store with metadata filtering
    """

    def __init__(self, db_path: str = "data/vectordb"):
        """
        Initialize the vector store

        Args:
            db_path: Path to the vector database directory
        """
        self.db_path = Path(db_path)
        self.db_path.mkdir(parents=True, exist_ok=True)

        self.faiss_index_path = self.db_path / "index.faiss"
        self.sqlite_path = self.db_path / "metadata.db"

        self.index: Optional[faiss.Index] = None

```

```
self.dimension = 384
```

```
self._init_sqlite()
```

```
def _init_sqlite(self):
```

```
    """Initialize SQLite database for metadata"""
```

```
    conn = sqlite3.connect(str(self.sqlite_path))
```

```
    cursor = conn.cursor()
```

```
    cursor.execute("""
```

```
        CREATE TABLE IF NOT EXISTS chunks (
```

```
            id TEXT PRIMARY KEY,
```

```
            text TEXT NOT NULL,
```

```
            metadata TEXT NOT NULL,
```

```
            embedding_id INTEGER UNIQUE,
```

```
            doc_id TEXT,
```

```
            created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
```

```
        )
```

```
    """)
```

```
    cursor.execute("""
```

```
        CREATE INDEX IF NOT EXISTS idx_doc_id ON chunks(doc_id)
```

```
    """)
```

```
    cursor.execute("""
```

```
        CREATE INDEX IF NOT EXISTS idx_embedding_id ON chunks(embedding_id)
```

```
    """)
```

```
    conn.commit()
```

```
    conn.close()
```

```

def _get_connection(self):
    """Get SQLite connection"""
    return sqlite3.connect(str(self.sqlite_path))

def _get_faiss_index(self) -> faiss.Index:
    """Get or create FAISS index"""
    if self.index is None:
        if self.faiss_index_path.exists():
            logger.info(f"Loading FAISS index from {self.faiss_index_path}")
            self.index = faiss.read_index(str(self.faiss_index_path))
            self.dimension = self.index.d
        else:
            logger.info("Creating new FAISS index")
            self.index = faiss.IndexFlatL2(self.dimension)
    return self.index

def _save_faiss_index(self):
    """Save FAISS index to disk"""
    if self.index is not None:
        logger.info(f"Saving FAISS index to {self.faiss_index_path}")
        faiss.write_index(self.index, str(self.faiss_index_path))

def _generate_chunk_id(self, text: str, metadata: Dict[str, Any]) -> str:
    """Generate unique ID for a chunk"""
    content = text + json.dumps(metadata, sort_keys=True)
    return hashlib.sha256(content.encode()).hexdigest()[:16]

def add_chunks(

```

```
self,  
chunks: List[str],  
embeddings: np.ndarray,  
metadata_list: List[Dict[str, Any]],  
doc_id: Optional[str] = None
```

```
) -> List[str]:
```

```
"""
```

Add chunks to the vector store

Args:

chunks: List of text chunks

embeddings: numpy array of embeddings

metadata_list: List of metadata dictionaries

doc_id: Document ID for grouping

Returns:

List of chunk IDs

```
"""
```

```
if len(chunks) != len(embeddings) or len(chunks) != len(metadata_list):
```

```
    raise ValueError("Chunks, embeddings, and metadata must have the same  
length")
```

```
index = self._get_faiss_index()
```

```
conn = self._get_connection()
```

```
cursor = conn.cursor()
```

```
chunk_ids = []
```

```
start_id = index.ntotal
```



```
for i, (chunk, embedding, metadata) in enumerate(zip(chunks, embeddings,
metadata_list)):
```

```
    # Generate chunk ID
```

```
    chunk_id = self._generate_chunk_id(chunk, metadata)
```

```
    # Check if chunk already exists (idempotent)
```

```
    cursor.execute("SELECT id FROM chunks WHERE id = ?", (chunk_id,))
```

```
    if cursor.fetchone():
```

```
        logger.debug(f"Chunk {chunk_id} already exists, skipping")
```

```
        continue
```

```
    # Add to FAISS
```

```
    embedding_np = embedding.astype(np.float32).reshape(1, -1)
```

```
    index.add(embedding_np)
```

```
    embedding_id = start_id + i
```

```
    # Add to SQLite
```

```
    metadata_json = json.dumps(metadata)
```

```
    cursor.execute("""
```

```
        INSERT INTO chunks (id, text, metadata, embedding_id, doc_id)
```

```
        VALUES (?, ?, ?, ?, ?)
```

```
    """, (chunk_id, chunk, metadata_json, embedding_id, doc_id))
```

```
    chunk_ids.append(chunk_id)
```

```
conn.commit()
```

```
conn.close()
```

```
self._save_faiss_index()
```

```
logger.info(f"Added {len(chunk_ids)} chunks to vector store")
return chunk_ids
```

```
def search(
    self,
    query_embedding: np.ndarray,
    k: int = 5,
    filter_metadata: Optional[Dict[str, Any]] = None
) -> List[Dict[str, Any]]:
```

```
"""
```

Search for similar chunks

Args:

query_embedding: Query embedding
k: Number of results to return
filter_metadata: Optional metadata filter

Returns:

List of search results with text, metadata, and scores

```
"""
```

```
index = self._get_faiss_index()
```

```
if index.ntotal == 0:
```

```
    return []
```

```
# Search FAISS
```

```
query_np = query_embedding.astype(np.float32).reshape(1, -1)
```

```
distances, indices = index.search(query_np, k)
```

```

# Get results from SQLite
conn = self._get_connection()
cursor = conn.cursor()

results = []
for dist, idx in zip(distances[0], indices[0]):
    if idx == -1:
        continue

    cursor.execute("""
        SELECT id, text, metadata, doc_id
        FROM chunks
        WHERE embedding_id = ?
    """, (int(idx),))

    row = cursor.fetchone()
    if row is None:
        continue

    chunk_id, text, metadata_json, doc_id = row
    metadata = json.loads(metadata_json)

# Apply metadata filter
if filter_metadata:
    matches = all(metadata.get(k) == v for k, v in filter_metadata.items())
    if not matches:
        continue

```

```

results.append({
    "id": chunk_id,
    "text": text,
    "metadata": metadata,
    "doc_id": doc_id,
    "score": float(1.0 - dist) # Convert L2 distance to similarity
})

conn.close()

# Sort by score (descending)
results.sort(key=lambda x: x["score"], reverse=True)

return results[:k]

def count(self) -> int:
    """Get total number of chunks"""
    index = self._get_faiss_index()
    return index.ntotal

def delete_document(self, doc_id: str):
    """Delete all chunks belonging to a document"""
    # This is a simplified implementation
    # FAISS doesn't support deletion directly, so we rebuild
    logger.warning(f"Deleting document {doc_id} requires rebuilding the index")

    conn = self._get_connection()
    cursor = conn.cursor()

```

```

# Get embedding IDs to delete
cursor.execute("SELECT embedding_id FROM chunks WHERE doc_id = ?",
(doc_id,))
embedding_ids = [row[0] for row in cursor.fetchall()]

if not embedding_ids:
    conn.close()
    return

# Delete from SQLite
cursor.execute("DELETE FROM chunks WHERE doc_id = ?", (doc_id,))
conn.commit()
conn.close()

# Rebuild FAISS index
self._rebuild_index()

logger.info(f"Deleted document {doc_id} with {len(embedding_ids)} chunks")

def _rebuild_index(self):
    """Rebuild FAISS index from SQLite data"""
    conn = self._get_connection()
    cursor = conn.cursor()

    cursor.execute("SELECT embedding_id, text, metadata FROM chunks ORDER BY
embedding_id")
    rows = cursor.fetchall()

    if not rows:
        self.index = faiss.IndexFlatL2(self.dimension)

```

```
self._save_faiss_index()
conn.close()
return
```

```
# Get all embeddings by reconstructing from IDs
# For simplicity, we'll just recreate the index
# In production, you'd need to store embeddings properly
self.index = faiss.IndexFlatL2(self.dimension)

# Add back all chunks
for i, row in enumerate(rows):
    # We need the original embedding, but we don't store it
    # This is a placeholder - in practice you'd store embeddings
    pass
```

```
conn.close()
self._save_faiss_index()
```

```
import os
from typing import List, Dict, Any, Optional
from pathlib import Path
import hashlib
import logging
from langchain.document_loaders import PyPDFLoader, UnstructuredHTMLLoader,
TextLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter

logger = logging.getLogger(__name__)
```

```
class DocumentIngester:
```

```
    """
```

```
    Ingests documents and prepares them for vector storage
```

```
    """
```

```
    def __init__(
```

```
        self,
```

```
        chunk_size: int = 512,
```

```
        chunk_overlap: int = 50,
```

```
        vector_store = None,
```

```
        embedding_generator = None
```

```
    ):
```

```
        """
```

```
        Initialize the document ingester
```

```
    Args:
```

```
        chunk_size: Size of text chunks
```

```
        chunk_overlap: Overlap between chunks
```

```
        vector_store: Vector store instance
```

```
        embedding_generator: Embedding generator instance
```

```
    """
```

```
        self.chunk_size = chunk_size
```

```
        self.chunk_overlap = chunk_overlap
```

```
        self.vector_store = vector_store
```

```
        self.embedding_generator = embedding_generator
```

```
        self.text_splitter = RecursiveCharacterTextSplitter(
```

```
            chunk_size=chunk_size,
```

```
            chunk_overlap=chunk_overlap,
```

```
        separators=["\\n\\n", "\\n", ".", " ", ""],
        length_function=len
    )
```

```
self.processed_docs = set()
```

```
def ingest_file(self, file_path: str, metadata: Optional[Dict[str, Any]] = None) -> int:
```

```
    """
```

```
    Ingest a single file
```

```
    Args:
```

```
        file_path: Path to the file
```

```
        metadata: Additional metadata
```

```
    Returns:
```

```
        Number of chunks ingested
```

```
    """
```

```
    file_path = Path(file_path)
```

```
    if not file_path.exists():
```

```
        raise FileNotFoundError(f"File not found: {file_path}")
```

```
    # Load document
```

```
    docs = self._load_document(file_path)
```

```
    if not docs:
```

```
        logger.warning(f"No content extracted from {file_path}")
```

```
    return 0
```



```
# Generate document ID
```

```
doc_id = self._generate_doc_id(file_path)
```

```
# Chunk documents
```

```
chunks = []
```

```
for doc in docs:
```

```
    doc_chunks = self.text_splitter.split_text(doc.page_content)
```

```
    chunks.extend(doc_chunks)
```

```
if not chunks:
```

```
    logger.warning(f"No chunks generated from {file_path}")
```

```
    return 0
```

```
# Prepare metadata
```

```
base_metadata = {
```

```
    "source": str(file_path),
```

```
    "filename": file_path.name,
```

```
    "doc_type": file_path.suffix[1:],
```

```
    "doc_id": doc_id
```

```
}
```

```
if metadata:
```

```
    base_metadata.update(metadata)
```

```
# Generate embeddings
```

```
embeddings = self.embedding_generator.embed(chunks)
```

```
# Add to vector store
```

```
metadata_list = [{**base_metadata, "chunk_index": i} for i in range(len(chunks))]
```

```
chunk_ids = self.vector_store.add_chunks(
```

```
        chunks=chunks,  
        embeddings=embeddings,  
        metadata_list=metadata_list,  
        doc_id=doc_id  
    )
```

```
self.processed_docs.add(doc_id)
```

```
logger.info(f"Ingested {len(chunk_ids)} chunks from {file_path}")  
return len(chunk_ids)
```

```
def ingest_directory(self, directory_path: str, metadata: Optional[Dict[str, Any]] = None)  
-> int:
```

```
    """
```

```
    Ingest all supported files in a directory
```

```
    Args:
```

```
        directory_path: Path to the directory
```

```
        metadata: Additional metadata
```

```
    Returns:
```

```
        Total number of chunks ingested
```

```
    """
```

```
    directory_path = Path(directory_path)
```

```
    if not directory_path.exists() or not directory_path.is_dir():
```

```
        raise ValueError(f"Directory not found: {directory_path}")
```

```
    total_chunks = 0
```

```
supported_extensions = [".pdf", ".html", ".htm", ".md", ".txt"]
```

```
for file_path in directory_path.rglob("*"):
    if file_path.suffix.lower() in supported_extensions:
        try:
            chunks = self.ingest_file(str(file_path), metadata)
            total_chunks += chunks
        except Exception as e:
            logger.error(f"Error ingesting {file_path}: {e}")
```

```
return total_chunks
```

```
def _load_document(self, file_path: Path) -> List:
```

```
    """
```

Load a document based on its extension

Args:

file_path: Path to the file

Returns:

List of document objects

```
    """
```

```
    suffix = file_path.suffix.lower()
```

```
    try:
```

```
        if suffix == ".pdf":
```

```
            loader = PyPDFLoader(str(file_path))
```

```
            return loader.load()
```

```
        elif suffix in [".html", ".htm"]:
```

```

        loader = UnstructuredHTMLLoader(str(file_path))
        return loader.load()
    elif suffix in [".md", ".txt"]:
        loader = TextLoader(str(file_path))
        return loader.load()
    else:
        logger.warning(f"Unsupported file type: {suffix}")
        return []
except Exception as e:
    logger.error(f"Error loading {file_path}: {e}")
    return []

```

```

def _generate_doc_id(self, file_path: Path) -> str:
    """Generate a unique document ID"""
    content = str(file_path.absolute()) + str(file_path.stat().st_mtime)
    return hashlib.sha256(content.encode()).hexdigest()[:16]

```

```

def get_processed_docs(self) -> List[str]:
    """Get list of processed document IDs"""
    return list(self.processed_docs)

```

```

from typing import List, Dict, Any, Optional
import logging

```

```

logger = logging.getLogger(__name__)

```

```

class Retriever:

```

```

    """

```

```

    Retrieves relevant chunks from the vector store

```

"""

```
def __init__(self, vector_store, embedding_generator, k: int = 5):
```

"""

Initialize the retriever

Args:

vector_store: Vector store instance

embedding_generator: Embedding generator instance

k: Default number of results to retrieve

"""

```
self.vector_store = vector_store
```

```
self.embedding_generator = embedding_generator
```

```
self.k = k
```

```
def retrieve(
```

```
    self,
```

```
    query: str,
```

```
    k: Optional[int] = None,
```

```
    filter_metadata: Optional[Dict[str, Any]] = None
```

```
) -> List[Dict[str, Any]]:
```

"""

Retrieve relevant chunks for a query

Args:

query: Query string

k: Number of results to retrieve

filter_metadata: Optional metadata filter

Returns:

List of retrieved chunks with scores

"""

if k is None:

k = self.k

Generate query embedding

query_embedding = self.embedding_generator.embed_single(query)

Search vector store

```
results = self.vector_store.search(
    query_embedding=query_embedding,
    k=k,
    filter_metadata=filter_metadata
)
```

logger.info(f"Retrieved {len(results)} chunks for query: {query[:50]}...")

return results

def format_context(self, results: List[Dict[str, Any]]) -> str:

"""

Format retrieved chunks as context for LLM

Args:

results: List of retrieved chunks

Returns:

Formatted context string

```
"""
```

```
if not results:
```

```
    return "No relevant context found."
```

```
context_parts = []
```

```
for i, result in enumerate(results, 1):
```

```
    source = result.get("metadata", {}).get("source", "Unknown source")
```

```
    context_parts.append(f"[{i}] Source: {source}\n{result['text']}\n")
```

```
return "\n".join(context_parts)
```

```
import time
```

```
import logging
```

```
from typing import Dict, Any, Optional, List
```

```
from openai import OpenAI
```

```
import tiktoken
```

```
logger = logging.getLogger(__name__)
```

```
class RAGPipeline:
```

```
    """
```

```
    RAG pipeline with LLM integration
```

```
    """
```

```
    def __init__(
```

```
        self,
```

```
        retriever,
```

```
        llm_model: str = "gpt-3.5-turbo",
```

```
        openai_api_key: Optional[str] = None,
```

```

    max_tokens: int = 500
):
    """
    Initialize the RAG pipeline

    Args:
        retriever: Retriever instance
        llm_model: LLM model to use
        openai_api_key: OpenAI API key
        max_tokens: Maximum tokens in response
    """

    self.retriever = retriever
    self.llm_model = llm_model
    self.max_tokens = max_tokens
    self.openai_client = None

    if openai_api_key:
        self.openai_client = OpenAI(api_key=openai_api_key)
    else:
        logger.warning("OpenAI API key not provided. LLM features disabled.")

    self.token_encoder = tiktoken.encoding_for_model("gpt-3.5-turbo")

def answer(
    self,
    query: str,
    k: Optional[int] = None,
    filter_metadata: Optional[Dict[str, Any]] = None
) -> Dict[str, Any]:

```


"""

Answer a query using RAG

Args:

query: User query

k: Number of chunks to retrieve

filter_metadata: Optional metadata filter

Returns:

Dictionary with answer and metadata

"""

start_time = time.time()

Retrieve relevant chunks

retrieval_start = time.time()

chunks = self.retriever.retrieve(query, k=k, filter_metadata=filter_metadata)

retrieval_time = time.time() - retrieval_start

Format context

context = self.retriever.format_context(chunks)

Generate answer

answer_start = time.time()

answer, tokens_used = self._generate_answer(query, context, chunks)

answer_time = time.time() - answer_start

total_time = time.time() - start_time

Log metrics

```
logger.info(f"Query: {query[:50]}...")
logger.info(f"Retrieved {len(chunks)} chunks in {retrieval_time*1000:.2f}ms")
logger.info(f"Answer generated in {answer_time*1000:.2f}ms")
logger.info(f"Total time: {total_time*1000:.2f}ms")
logger.info(f"Tokens used: {tokens_used}")
```

```
return {
    "query": query,
    "answer": answer,
    "context": context,
    "chunks": chunks,
    "metrics": {
        "retrieval_time_ms": retrieval_time * 1000,
        "answer_time_ms": answer_time * 1000,
        "total_time_ms": total_time * 1000,
        "chunk_count": len(chunks),
        "tokens_used": tokens_used
    }
}
```

```
def _generate_answer(
    self,
    query: str,
    context: str,
    chunks: List[Dict[str, Any]]
) -> tuple[str, int]:
    """
```

Generate an answer using the LLM

Args:

query: User query

context: Formatted context

chunks: Retrieved chunks

Returns:

Tuple of (answer, tokens_used)

"""

if not self.openai_client:

return self._generate_fallback_answer(query, context), 0

if not chunks:

return self._handle_no_context(query), 0

Construct prompt

prompt = f"""You are a helpful assistant that answers questions based on the provided context.

Context:

{context}

Question: {query}

Instructions:

1. Answer based ONLY on the context provided.
2. If the context doesn't contain the answer, say "I don't have enough information to answer this question."
3. Cite the sources you use (e.g., [1], [2]).
4. Be concise but thorough.

Answer: ""

try:

```
    response = self.openai_client.chat.completions.create(
        model=self.llm_model,
        messages=[
            {"role": "system", "content": "You are a helpful assistant that answers
questions based on provided context."},
            {"role": "user", "content": prompt}
        ],
        max_tokens=self.max_tokens,
        temperature=0.1
    )
```

```
    answer = response.choices[0].message.content
```

```
    tokens_used = response.usage.total_tokens
```

```
    return answer, tokens_used
```

except Exception as e:

```
    logger.error(f"Error generating answer: {e}")
```

```
    return self._generate_fallback_answer(query, context), 0
```

```
def _handle_no_context(self, query: str) -> str:
```

```
    """Handle case with no relevant context"""
```

```
    return "I don't have enough information to answer this question. The document
corpus doesn't contain relevant context."
```

```
def _generate_fallback_answer(self, query: str, context: str) -> str:
```

```
    """Generate a fallback answer without LLM"""
```

```
if not context or "No relevant context" in context:
```

```
    return "I don't have enough information to answer this question."
```

```
    return "Based on the provided context, I can answer this question. (LLM API key not configured)"
```

```
import json
```

```
import numpy as np
```

```
from typing import List, Dict, Any, Tuple
```

```
from sklearn.metrics.pairwise import cosine_similarity
```

```
import logging
```

```
logger = logging.getLogger(__name__)
```

```
class Evaluator:
```

```
    """
```

```
    Evaluate RAG system performance
```

```
    """
```

```
    def __init__(self, rag_pipeline, embedding_generator):
```

```
        """
```

```
        Initialize the evaluator
```

```
        Args:
```

```
            rag_pipeline: RAG pipeline instance
```

```
            embedding_generator: Embedding generator instance
```

```
        """
```

```
        self.rag_pipeline = rag_pipeline
```

```
        self.embedding_generator = embedding_generator
```

```
self.results = []
```

```
def evaluate_questions(self, questions: List[Dict[str, Any]]) -> Dict[str, Any]:
```

```
    """
```

```
    Evaluate the RAG system on a set of questions
```

```
    Args:
```

```
        questions: List of question dictionaries with keys:
```

- question: The question text
- gold_answer: Optional gold answer
- relevant_chunks: Optional list of relevant chunk IDs
- category: Optional category

```
    Returns:
```

```
        Evaluation metrics
```

```
    """
```

```
    metrics = {
```

```
        "retrieval": [],
```

```
        "answer": [],
```

```
        "latency": []
```

```
    }
```

```
    for q in questions:
```

```
        question_text = q["question"]
```

```
        # Get RAG response
```

```
        response = self.rag_pipeline.answer(question_text)
```

```
# Store results
```

```
result = {  
    "question": question_text,  
    "answer": response["answer"],  
    "chunks": response["chunks"],  
    "metrics": response["metrics"],  
    "gold_answer": q.get("gold_answer", ""),  
    "relevant_chunks": q.get("relevant_chunks", []),  
    "category": q.get("category", "general")  
}  
self.results.append(result)
```

```
# Compute retrieval metrics if relevant chunks are known
```

```
if q.get("relevant_chunks"):  
    retrieval_metrics = self._compute_retrieval_metrics(  
        retrieved_chunks=response["chunks"],  
        relevant_chunk_ids=q["relevant_chunks"]  
    )  
    metrics["retrieval"].append(retrieval_metrics)
```

```
# Compute answer metrics
```

```
if q.get("gold_answer"):  
    answer_metrics = self._compute_answer_metrics(  
        generated=response["answer"],  
        gold=q["gold_answer"],  
        context=response["context"]  
    )  
    metrics["answer"].append(answer_metrics)
```

```

# Latency metrics
metrics["latency"].append({
    "total_time_ms": response["metrics"]["total_time_ms"],
    "retrieval_time_ms": response["metrics"]["retrieval_time_ms"],
    "answer_time_ms": response["metrics"]["answer_time_ms"],
    "chunk_count": response["metrics"]["chunk_count"]
})

# Aggregate metrics
return self._aggregate_metrics(metrics)

def _compute_retrieval_metrics(
    self,
    retrieved_chunks: List[Dict[str, Any]],
    relevant_chunk_ids: List[str]
) -> Dict[str, float]:
    """Compute retrieval metrics"""
    # Get retrieved IDs
    retrieved_ids = [c["id"] for c in retrieved_chunks]

    # Hit rate (recall@k)
    hits = sum(1 for rid in retrieved_ids if rid in relevant_chunk_ids)
    hit_rate = hits / min(len(relevant_chunk_ids), len(retrieved_ids)) if
relevant_chunk_ids else 0

    # Precision@k
    precision = hits / len(retrieved_ids) if retrieved_ids else 0

    # MRR (Mean Reciprocal Rank)

```



```

mrr = 0
for i, rid in enumerate(retrieved_ids, 1):
    if rid in relevant_chunk_ids:
        mrr = 1 / i
        break

# nDCG@k (simplified)
ndcg = 0
relevance = [1 if rid in relevant_chunk_ids else 0 for rid in retrieved_ids]
if relevance:
    dcg = sum(rel / np.log2(i + 2) for i, rel in enumerate(relevance))
    idcg = sum(1 / np.log2(i + 2) for i in range(min(len(relevant_chunk_ids),
len(retrieved_ids))))
    ndcg = dcg / idcg if idcg > 0 else 0

return {
    "hit_rate": hit_rate,
    "precision": precision,
    "mrr": mrr,
    "ndcg": ndcg
}

def _compute_answer_metrics(
    self,
    generated: str,
    gold: str,
    context: str
) -> Dict[str, float]:
    """Compute answer quality metrics"""

```

```
# Token overlap (simplified EM/F1)
```

```
gold_tokens = set(gold.lower().split())
```

```
gen_tokens = set(generated.lower().split())
```

```
# Exact Match
```

```
em = 1.0 if generated.strip() == gold.strip() else 0.0
```

```
# F1
```

```
intersection = gold_tokens & gen_tokens
```

```
precision = len(intersection) / len(gen_tokens) if gen_tokens else 0
```

```
recall = len(intersection) / len(gold_tokens) if gold_tokens else 0
```

```
f1 = 2 * precision * recall / (precision + recall) if (precision + recall) > 0 else 0
```

```
return {
```

```
    "em": em,
```

```
    "f1": f1,
```

```
    "precision": precision,
```

```
    "recall": recall
```

```
}
```

```
def _aggregate_metrics(self, metrics: Dict[str, List]) -> Dict[str, Any]:
```

```
    """Aggregate metrics across all questions"""
```

```
    aggregated = {
```

```
        "retrieval": {},
```

```
        "answer": {},
```

```
        "latency": {},
```

```
        "summary": {}
```

```
}
```

```
# Aggregation for retrieval metrics
```

```
if metrics["retrieval"]:
```

```
    for key in metrics["retrieval"][0].keys():
```

```
        values = [m[key] for m in metrics["retrieval"] if key in m]
```

```
        if values:
```

```
            aggregated["retrieval"][key] = {
```

```
                "mean": np.mean(values),
```

```
                "std": np.std(values),
```

```
                "min": np.min(values),
```

```
                "max": np.max(values)
```

```
            }
```

```
# Aggregation for answer metrics
```

```
if metrics["answer"]:
```

```
    for key in metrics["answer"][0].keys():
```

```
        values = [m[key] for m in metrics["answer"] if key in m]
```

```
        if values:
```

```
            aggregated["answer"][key] = {
```

```
                "mean": np.mean(values),
```

```
                "std": np.std(values),
```

```
                "min": np.min(values),
```

```
                "max": np.max(values)
```

```
            }
```

```
# Aggregation for latency metrics
```

```
if metrics["latency"]:
```

```
    for key in metrics["latency"][0].keys():
```

```
        values = [m[key] for m in metrics["latency"] if key in m]
```

```
        if values:
```

```

        aggregated["latency"][key] = {
            "mean": np.mean(values),
            "std": np.std(values),
            "min": np.min(values),
            "max": np.max(values),
            "p50": np.percentile(values, 50),
            "p95": np.percentile(values, 95)
        }

```

Summary

```

        aggregated["summary"] = {
            "total_questions": len(metrics["latency"]),
            "avg_total_time_ms": np.mean([m["total_time_ms"] for m in metrics["latency"]]),
            "avg_chunk_count": np.mean([m["chunk_count"] for m in metrics["latency"]]),
            "avg_tokens_used": np.mean([m.get("tokens_used", 0) for m in
metrics["latency"]])
        }

```

return aggregated

def save_results(self, output_path: str):

"""Save evaluation results to file"""

output = {

"results": self.results,

"aggregated": self._aggregate_metrics({}) # Placeholder

}

with open(output_path, "w") as f:

json.dump(output, f, indent=2)

```
logger.info(f"Results saved to {output_path}")
```

```
from flask import Flask, request, jsonify, render_template
```

```
from flask_cors import CORS
```

```
import logging
```

```
from typing import Dict, Any
```

```
import os
```

```
from .config import config
```

```
from .embeddings import EmbeddingGenerator
```

```
from .vector_store import VectorStore
```

```
from .retrieval import Retriever
```

```
from .rag import RAGPipeline
```

```
from .ingestion import DocumentIngester
```

```
from .evaluation import Evaluator
```

```
# Configure logging
```

```
logging.basicConfig(
```

```
    level=getattr(logging, config.LOG_LEVEL),
```

```
    format='%(asctime)s - %(name)s - %(levelname)s - %(message)s'
```

```
)
```

```
logger = logging.getLogger(__name__)
```

```
# Initialize components
```

```
embedding_generator
```